

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

 Picture/BKU.png

COMPUTER NETWORKS (HPMR)

Extension: Design ZeroMQ

Instructor: NGUYEN PHUONG DUY

Students: Mai Xuan Nhut - 2312549
Nguyen Viet Hoang - 2311066
Nguyen Chi Thanh - 2313078
Le Duc Tai - 2312995

Class ID: CO3094 (A01)

HO CHI MINH CITY, 2026

Contents

Abstract

This report details the architectural transition of a chat backend from a synchronous HTTP-based model to a high-performance, brokerless asynchronous system using ZeroMQ. By implementing a PUSH/PULL messaging pattern, we achieved significant reductions in latency and protocol overhead. The design emphasizes the "Smart Endpoint, Dumb Network" philosophy, leveraging lock-free algorithms and the Actor model to ensure linear scalability on multi-core environments while supporting both TCP and intra-process communication.

1 Problem Statement

The original implementation relied on HTTP request-response semantics, which proved inefficient for real-time chat:

- **High Overhead:** Every message requires HTTP header parsing and multiple network traversals.
- **Blocking I/O:** Traditional HTTP servers often struggle with massive concurrency due to blocking calls in the critical path.
- **Centralized Bottleneck:** The use of a central broker or web controller induces a penalty in throughput and latency.

2 Implemented Architecture

The implementation is intentionally small and follows one message-oriented path end to end. The application itself owns the routing logic, and ZeroMQ provides the transport and socket primitives.

2.1 Launcher and Entry Point

The launcher in `start_chatapp.py` exposes ZeroMQ-specific flags instead of HTTP-specific ones.

- `-bind-ip` and `-bind-port` define where the server PULL socket listens.
- `-server-endpoint` allows the server to be started directly with a `tcp://` address.
- The launcher simply forwards the selected bind address to `create_chatapp()`.

This is the first practical change from the HTTP version: the server is launched as a ZeroMQ message endpoint, not as a web application.

2.2 Server-Side Message Handling

The backend in `apps/chatapp.py` is responsible for three concrete tasks:

1. bind one PULL socket for incoming messages,
2. maintain an in-memory registry from user name to receiving endpoint,
3. forward messages to the correct destination using PUSH sockets.

The implementation idea is simple:

- When the server receives a **register** message, it stores the client's address in the registry.
- When it receives a **send** message, it looks up the target user and forwards the payload only to that user.
- When it receives a **broadcast** message, it iterates over the registry and forwards the same payload to all registered users.

This design keeps the message path short and avoids translating chat traffic into HTTP requests.

2.3 Client-Side Message Handling

The client in `apps/zmq_chat_client.py` is split into two parts:

- a background receiver thread that binds a local PULL socket and prints incoming chat messages,
- a sender loop that connects a PUSH socket to the server and sends register/direct/broadcast commands.

The idea behind this code is to separate inbound and outbound traffic cleanly. The receiver thread handles incoming payloads independently, while the foreground loop remains interactive for user input.

2.4 Message Flow

The runtime flow is implemented directly in the socket logic:

1. The client binds its local receive socket.
2. The client sends a **register** message to the server.
3. The server stores the client's endpoint.
4. For direct messaging, the sender sends **send** and the server forwards the payload to one destination.
5. For broadcast, the sender sends **broadcast** and the server forwards the payload to all destinations.

2.5 Automated Smoke Test

The file `benchmarks/zmq_chat_smoke.py` shows how the implementation is verified in code.

- It starts the server as a subprocess.
- It creates two clients with separate receive ports.
- It sends registration frames, a direct message, and a broadcast message.
- It asserts that the correct recipient sockets actually receive the expected payloads.

This test is important because it proves the solution works as a live ZeroMQ pipeline rather than just as a design sketch.

3 Interface Design Criteria

The submission is also evaluated by four practical interface criteria. The implementation and report address each one directly.

3.1 Interface Clarity

The message interface is intentionally small and easy to read. Every message uses a simple JSON schema with a **type** field and a small set of required attributes:

- **register**: declares the client name and receive port,

- **send:** carries sender, recipient, and message content,
- **broadcast:** carries sender and message content.

This structure makes the connection flow easy to document and easy to test for all connection types.

3.2 Connection Efficiency

The design is efficient because it avoids HTTP parsing, request routing, and synchronous round trips. Instead:

- clients push messages asynchronously to the server,
- the server keeps an in-memory registry for direct lookup,
- delivery happens through ZeroMQ sockets with minimal protocol overhead,
- the same socket pattern works for both direct delivery and broadcast delivery.

This keeps latency low and resource usage modest, which is the main benefit of the ZeroMQ approach.

3.3 Error Handling

The implementation includes practical checks for malformed or incomplete data:

- the server rejects missing names and invalid receive ports during registration,
- malformed send messages are detected before forwarding,
- unknown recipients are reported by the server instead of crashing the process,
- the client validates the command syntax for direct and broadcast messages.

These checks keep the connection layer stable even when bad payloads are received.

3.4 Scalability Design

The architecture is scalable because each new client only adds one registry entry and one receive socket. There is no need to rewrite the protocol when load increases.

- The server can handle more clients by extending the in-memory registry.

- Direct messaging remains efficient because the server resolves the target in constant time.
- Broadcast delivery scales by iterating over the registered endpoints without adding a separate broker process.
- The same design can later be extended to more nodes, additional worker processes, or alternative transports such as `inproc`.

4 Validation and Benchmarking

Validation was performed using an automated smoke test covering registration, direct delivery, and broadcast functionality.

- Confirmed successful server startup with a ZeroMQ PULL endpoint.
- Verified that direct messages reach only the intended recipient without HTTP overhead.
- Demonstrated that broadcast messages are delivered asynchronously to all registered nodes.

5 AI Attribution

In accordance with university regulations, we disclose that AI (Gemini) was used for:

- Assisting with LaTeX formatting and technical English terminology.
- Summarizing architectural concepts from external technical documentation.
- Providing a template for the report structure and command-line usage examples.
- *Note: All design decisions and final code validation were conducted by the student group.*

6 Conclusion

This project successfully demonstrates the migration to an asynchronous brokerless architecture. By focusing on the critical path and utilizing lock-free concurrency, the system provides a robust foundation for low-latency, extreme-speed messaging that far exceeds the capabilities of standard synchronous HTTP models.